

TALLINN UNIVERSITY OF TECHNOLOGY
IT College

Marko Linde 176292IADB

CALCULATOR

Javascript (ICD0006)

Instructor: Andres Käver

Tallinn 2022

Table of Contents

1 Assignment	3
2 Overview	4
2.1 Calculator logic.....	4
2.2 Display.....	6
2.3 Functionality.....	7
2.3.1 Main.....	7
2.3.2 Evaluations after calculate command.....	7
2.3.3 Calculation command.....	7
Annex 1 – index.js	8

1 Assignment

Write simple calculator using Bootstrap (or any other css lib) based design and simple js functionality.

Provide basic functions - +, -, *, /, C

Write your calculator using an object oriented approach. Separate code between business logic and ui.

Please implement classical calculator UI - like you have in your mobile phone.

Nice to have - calculator is done in responsive design. Fits full-screen, both on desktop and on mobile.

Use modules/exports/imports.

UI has to be generated fully in JS.

Only allowed element inside <body> tag is bootstrap import and single custom js import.

Support multiple calculators on page in parallel (every calculator is separate from each other).

Implement UI for dynamic adding and removal

2 Overview

Index.html and index.css taken from calculator example . Index.js is written by document author.

2.1 Calculator logic

Basic idea comes from logic, that the functionality of the calculator can be expanded.

- All calculator clicks are saved in a long string only array which allows:
 - to display the entire equation to the user

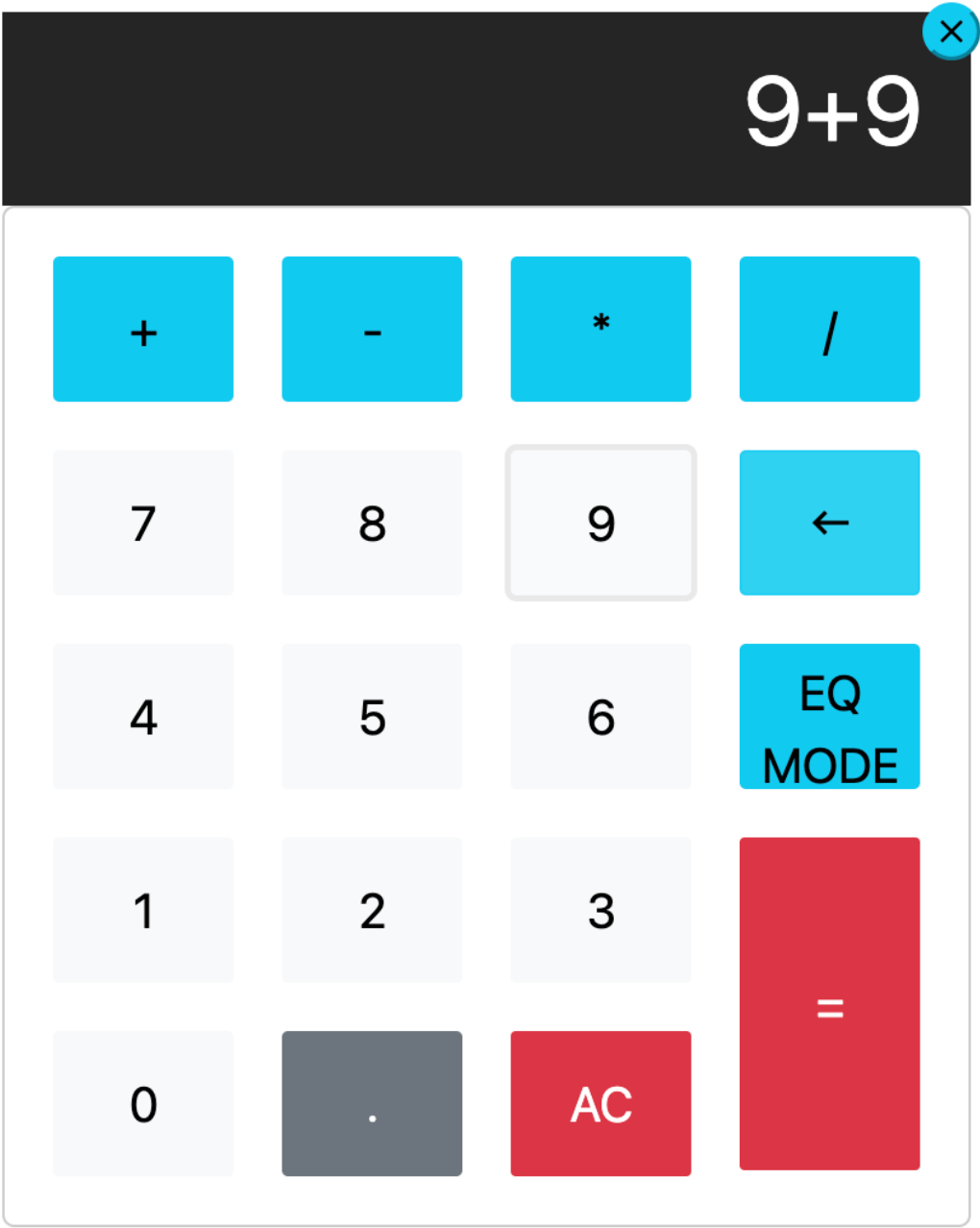


Drawing 1. Equation view

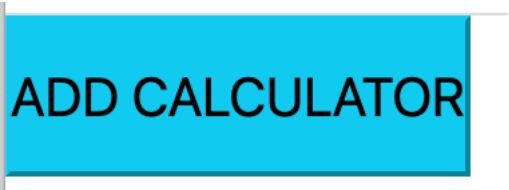
- to add undo button functionality
 - to add calculate only parts of the equation functionality
 - to add click on equation element and repair it directly functionality
 - to make calculator more scientific
- Calculator starts to calculate only when clicking the equal sign
- In EQ mode the calculations are done from left to right with rule: first multiply or divide then add or subtract
- In Not EQ mode the calculations are done from left to right
- Backspace removes last equation entry
- X mark closes calculator
- Calculator business logic with calculator module are injected into Calculator UI

- Index.html is segregated into navbar and main sections
- Calculator receives distinct ID (differentiates calculators) by card1.....n
- ID is generated by static class (id is not needed, but in case required later, it is kept)
- Close button is inside calculator module
- Calculator add but in navbar.

2.2 Display



Drawing 2. Calculator screen



Drawing 3. Add calculator button

2.3 Functionality

2.3.1 Main

- Elements are observed through node lists
- Html calculator is generated by module class
- Every button click value is inserted into equations array separately
- Equal button clicking calls calculation function
- AC button click calls overwriting of the previous array by empty array
- Backspace removes last equation entry
- EQ mode switches between two calculation modes
- X button closes calculator
- Add button on navbar adds calculator

2.3.2 Evaluations after calculate command

- Operators the start (except minus) and end equation are evaluated – that they don't exist
- Two operators straight are not allowed
- Two dots straight are not allowed
- Operator and dot are not allowed

2.3.3 Calculation command

- Number of operators is searched
- First calculation operator is searched from the equation and its index is returned.
- All numbers between operators are converted to full numbers
- Calculated value is inserted back into the equation
- New calculation command iteration will begin

Annex 1 – index.js

```
import { ModuleId } from "../public/moduleId";
/**
 * Class which groups together all html elements for calculator
 */
export class CalculatorModule{

    calculatorContent;
    calculatorScreen;
    calculatorKeyModule;
    closeKey;
    moduleId;

    constructor(){
        this.moduleId=ModuleId.getNextId();
        this.calculatorContent = document.createElement("div");
        this.calculatorContent.className = "calculator card"+this.moduleId;
        this.calculatorContent.style.display = 'inline-block';

        this.closeKey = this.calculatorKey('close','material-icons','btn-info','button-close')
        this.calculatorContent.appendChild(this.closeKey);

        this.calculatorScreen = this.setCalculatorScreen("calculator-screen","z-depth-1");
        this.calculatorContent.appendChild(this.calculatorScreen);

        this.calculatorKeyModule = this.setCalculatorKeyModule();
        ['+', '-', '*', '/'].forEach(operator =>{
            this.calculatorKeyModule.appendChild(this.calculatorKey(operator, 'btn-info', 'btn'));
        });
        ['7', '8', '9'].forEach(operator =>{
            this.calculatorKeyModule.appendChild(this.calculatorKey(operator, 'btn-light', 'waves-effect', 'btn'));
        });
    }
}
```



```

});
this.calculatorKeyModule.appendChild(this.calculatorKey('keyboard_backspace','operator','material-icons','btn-info','btn'));
['4','5','6'].forEach(operator =>{
    this.calculatorKeyModule.appendChild(this.calculatorKey(operator,'btn-light','waves-effect','btn'));
});
this.calculatorKeyModule.appendChild(this.calculatorKey('EQ MODE','operator','btn-info','btn'));
['1','2','3','0'].forEach(operator =>{
    this.calculatorKeyModule.appendChild(this.calculatorKey(operator,'btn-light','waves-effect','btn'));
});
this.calculatorKeyModule.appendChild(this.calculatorKey('.', 'decimal', 'function', 'btn-secondary', 'btn'));
this.calculatorKeyModule.appendChild(this.calculatorKey('AC', 'all-clear', 'function', 'btn-danger', 'btn-sm', 'btn'));
this.calculatorKeyModule.appendChild(this.calculatorKey('=', 'equal-sign', 'operator', 'btn-danger', 'btn-sm', 'btn'));

this.calculatorContent.append(this.calculatorKeyModule);
}

freeModuleId(){
    ModuleId.freeId(this.moduleId);
}

setCalculatorScreen(...cssClasses){
    let calculatorScreen = document.createElement("input");
    calculatorScreen.type = "text";

    cssClasses.forEach(cssClass => {
        calculatorScreen.classList.add(cssClass);
    });

    calculatorScreen.value = "";
    calculatorScreen.disabled = true;
    return calculatorScreen;
}

```

```
setCalculatorKeyModule(){
  let calcModule = document.createElement("div");
  calcModule.className = "calculator-keys"
  return calcModule;
}

calculatorKey(text,...cssClasses){
  let button = document.createElement("button");

  cssClasses.forEach(cssClass => {
    button.classList.add(cssClass);
  });

  button.value = text;
  button.textContent = text;
  return button;
}
}
```

```

import { CalculatorModule } from "../calculatorModule";
/**
 * Takes care of calculator user interface, which
 * relays the information from web page to calculation
 * and pack
 */
export class CalculatorUI{

  constructor(calculus,module){
    this.calculus = calculus;
    this.module = module;

    module.closeKey.onclick = (() => {
      this.module.freeModuleId();
      this.module.calculatorContent.remove();
    });

    module.calculatorKeyModule.childNodes.forEach(child =>{
      if (child){
        child.onclick = (() => {
          this.action(child.getAttribute('value'));
          if (child.getAttribute('value') === 'EQ MODE'){
            child.style.color= (child.style.color === 'red') ? 'black' : 'red';
          }
        });
      }
    });
  }

  action = (action) => {
    this.calculus.actionCaller(action);
    this.refresh();
  }
}

```

```

//refreshes display
refresh = () => {
    this.module.calculatorScreen.value = this.calculus.getEquationState();
}
}

```

```

import { Equation } from "./equation";
/**
 * Businesslogic class to perform calculations and find errors
 * Todo: move errors to separate class
 */
export class Calculus {

    equation;
    equationState;
    operators;
    eqMode;

    constructor(){
        this.equation = new Equation();
        this.equationState = undefined;
        this.operators = ['+', '-', '*', '/'];
        this.eqMode = 0;
    }
}

```

```

actionCaller = (action) =>{
  console.log(action);
  console.log(JSON.stringify(this.equation));

  if (!(this.findErrorAction(action))){
    switch (action) {
      case 'AC':
        this.equation.clear();
        break;
      case '=':
        this.equation.convertEquationToFullNumbers(this.operators);
        this.calculate();
        break;
      case 'keyboard_backspace':
        this.equation.removeLast();
        break;
      case 'EQ MODE':
        this.eqMode = this.setEqMode();
        console.log("Eqmode :" + JSON.stringify(this.eqMode));
        break;
      default:
        this.equation.addElement(action);
    }
  }
  this.equationState = this.equation.getConcatEquation();
}

//switches system between mathematical standard and linear calculation modes
setEqMode = () =>{
  return (this.eqMode === 1) ? 0 : 1;
}

```

```

//returns equation to screen
getEquationState = () =>{
    return this.equationState;
}

//find user entered problems
findErrorAction = (action) =>{
    let error = 0;
    if (this.equation.getConcatEquation().length === 0 && this.equation.getIndex(this.operators,action) > -1 &&
        action !== '-') error = 1;
    else if ((this.isPreviousEntryAnOperator(this.operators) || this.isPreviousEntryAnOperator(['.'])) &&
        action === '=') error = 1;
    else if ((this.isPreviousEntryAnOperator(this.operators) || this.isPreviousEntryAnOperator(['.'])) &&
        (this.equation.getIndex(this.operators,action) > -1 || action === '.')) error = 1;
    return error;
}

isPreviousEntryAnOperator = (operators) =>{
    return (this.equation.getIndex(operators,
        this.equation.equation[this.equation.equation.length-1]) > -1) ? 1 : 0;
}

calculate = () =>{
    while(this.equation.getNumberOfSymbolsInEquation(this.operators)){
        if (this.eqMode === 1) this.calculateIndexValue(this.getNextCalculationIndex());
        else this.calculateIndexValue(1);
    }
}

```

```

//calculate equation value on index operator with before and after numbers
//and inserts subcalculation back to equation
calculateIndexValue = (startIndex) =>{
  let value = NaN;

  switch(this.equation.getIndexValue(startIndex)) {
    case '*':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) *
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '/':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) /
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '+':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) +
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '-':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) -
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    default:
      value;
  }
  console.log('Index calculation value: ' + value);
  this.equation.changeEquation(startIndex,value.toString());
  console.log('Equation: ' + JSON.stringify(this.equation.equation));
}

```

```

//based on mathematical logic get first calcualtion operator
getNextCalculationIndex = () =>{
  let startIndex = NaN;

```

```

  if (((this.getEquationIndex('*') !== -1) && (this.getEquationIndex('*') < this.getEquationIndex('/')) ||

```

```

        ((this.getEquationIndex('/') === -1) && (this.getEquationIndex('*') > this.getEquationIndex('/'))))
            startIndex = this.getEquationIndex('*');
    else if (this.getEquationIndex('/') >= 1) startIndex = this.getEquationIndex('/');
    else if (this.getEquationIndex('+') >= 1) startIndex = this.getEquationIndex('+');
    else if (this.getEquationIndex('-') >= 1) startIndex = this.getEquationIndex('-');

    console.log('Starting with index: ' + startIndex);
    return startIndex;
}

getEquationIndex = (value) =>{
    return this.equation.getIndex(this.equation.equation, value);
}
}

```



```

/**
 * Equation is class to preserve user clicked
 * information and perform basic operations with
 * array equation
 */
export class Equation {

    equation = [];

    constructor(){
        this.equation = [];
    }

    clear(){
        this.equation = [];
        this.number= [];
        console.log(JSON.stringify(this.equation));
    }

    addElement = (elem) => {
        this.equation.push(elem);
        console.log(JSON.stringify(this.equation));
    }

    // all individual numeric elements between symbols are converted to full numbers
    convertEquationToFullNumbers = (operators) =>{
        let index=0;
        let number="";
        let compressedEquation=[];

        this.equation.forEach(elem =>{
            if (this.getIndex(operators,elem) > -1 && !(elem === '-' && index === 0)){
                compressedEquation.push(number);
                compressedEquation.push(elem);
            }
        });
    }
}

```

```

        number="";
    } else {
        if (number === "") number = elem;
        else number = number.concat(this.equation[index]);
    }
    index++;
});
if (number !== ""){
    compressedEquation.push(number);
}

this.equation = compressedEquation;
console.log(this.equation);
}

```

```

getNumberOfSymbolsInEquation = (operators) =>{
    let counter = 0;

    this.equation.forEach(elem => {
        if (this.getIndex(operators,elem) > -1) counter +=1;
    });

    return counter;
}

```

```

getIndex(operators,sign){
    return operators.findIndex((element) => element === sign);
}

```

```

getConcatEquation = () =>{
    return this.equation.join('');
}

```

```

getIndexValue = (index) => {
    return this.equation[index];
}

```

```
}

getPreviousValue = (index) => {
  return this.equation[index-1];
}

getNextValue = (index) => {
  return this.equation[index+1];
}

changeEquation = (index,value) =>{
  if (index >= 0) this.equation.splice(index - 1,3,value);
}

removeLast = () =>{
  this.equation.pop();
}
}
```

```

/**
 * Keeps distinct id-s.
 */
export class ModuleId {
  static idCounter = 0;
  static freeIds = [];

  static getNextId() {
    if (this.freeIds.length > 0) return this.freeIds.pop();
    else return ++this.idCounter;
  }

  static freeId(id) {
    this.freeIds.push(id);
  }
}

```

```

<html>
  <head>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">
    <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">
  </head>
  <body>
  </body>
</html>

```

```

"use strict";
import css from "../index.css"
import { CalculatorModule } from "../public/calculatorModule";
import { CalculatorUI } from "../public/calculatorUI";
import { Calculus } from "../public/calculus";

/**
 * Setting up WebPage areas
 */
let navBar = document.createElement('div');
let navBarButton = document.createElement('button');
navBarButton.textContent = 'ADD CALCULATOR';
navBarButton.classList = ('btn', 'btn-info');

navBar.appendChild(navBarButton);
document.body.appendChild(navBar);

let centerArea = document.createElement('div');

document.body.appendChild(centerArea);

function addCalculator(){
    navBarButton.onclick = function() {
        let calcMod = new CalculatorModule();
        centerArea.appendChild(calcMod.calculatorContent);
        new CalculatorUI(new Calculus(), calcMod);
    }
}

addCalculator();

```